



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Lab Programming 14

Alpha-Beta Pruning Algorithm for Gaming

Aim

To write a Python program to implement the Alpha-Beta Pruning algorithm for a simple game.

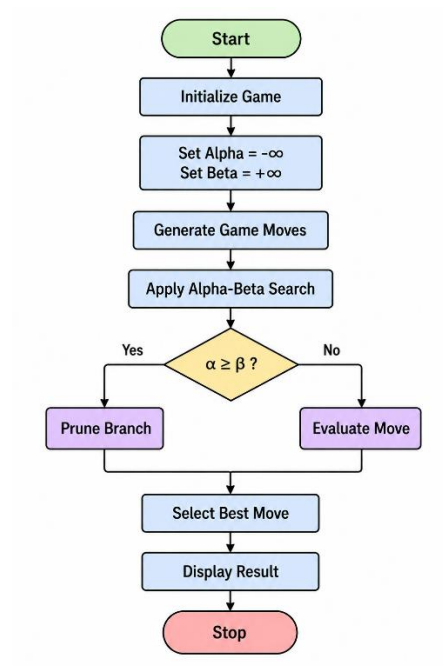
Objective

To implement Alpha-Beta pruning.

Algorithm

1. Start.
2. Create the game states.
3. Set $\alpha = -\infty$ and $\beta = +\infty$.
4. Evaluate possible moves using Minimax.
5. Prune branches when $\beta \leq \alpha$.
6. Select the best move.
7. Display the result.
8. Stop.

Flowchart



Code

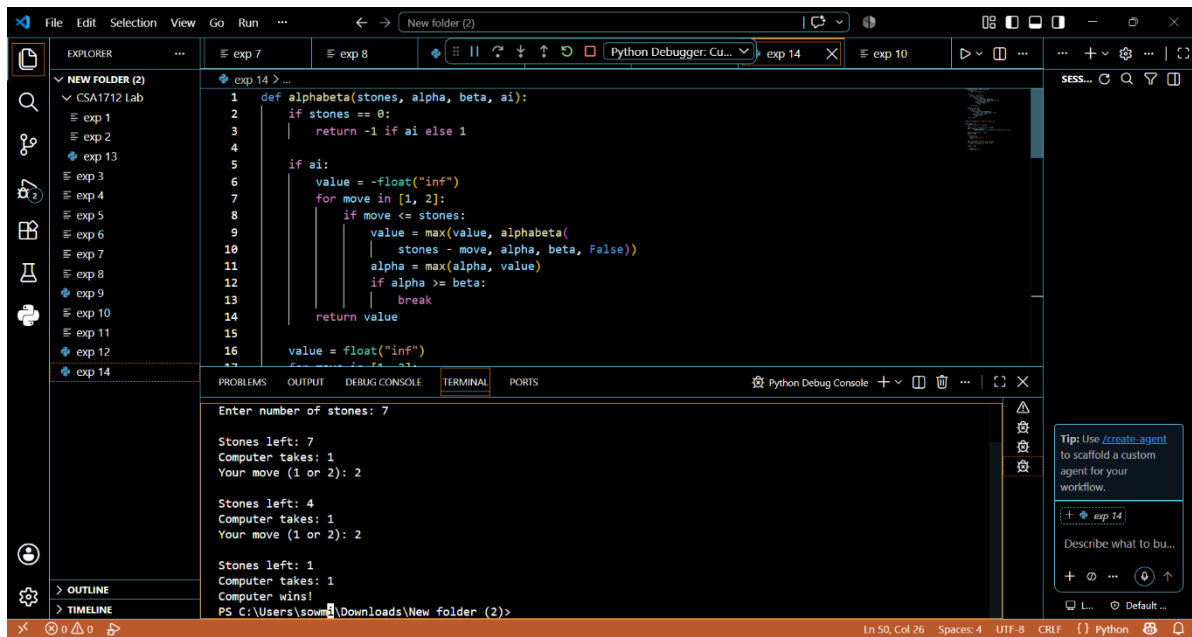
```
def alphabeta(stones, alpha, beta, ai):  
    if stones == 0:  
        return -1 if ai else 1  
    if ai:  
        value = -float("inf")  
        for move in [1, 2]:  
            if move <= stones:  
                value = max(value, alphabeta(  
                    stones - move, alpha, beta, False))  
                alpha = max(alpha, value)  
                if alpha >= beta:  
                    break  
        return value
```

```

value = float("inf")
for move in [1, 2]:
    if move <= stones:
        value = min(value, alphabeta(
            stones - move, alpha, beta, True))
        beta = min(beta, value)
        if alpha >= beta:
            break
    return value
stones = int(input("Enter number of stones: "))
while stones > 0:
    print("\nStones left:", stones)
    moves = [m for m in [1, 2] if m <= stones]
    move = max(moves, key=lambda m:
        alphabeta(stones - m, -float("inf"), float("inf"), False))
    print("Computer takes:", move)
    stones -= move
    if stones == 0:
        print("Computer wins!")
        break
    move = int(input("Your move (1 or 2): "))
    while move not in [1, 2] or move > stones:
        move = int(input("Enter 1 or 2: "))
    stones -= move
    if stones == 0:
        print("You win!")

```

Output



```
1 def alphabeta(stones, alpha, beta, ai):
2     if stones == 0:
3         return -1 if ai else 1
4
5     if ai:
6         value = -float("inf")
7         for move in [1, 2]:
8             if move <= stones:
9                 value = max(value, alphabeta(
10                     stones - move, alpha, beta, False))
11                 alpha = max(alpha, value)
12                 if alpha >= beta:
13                     break
14         return value
15
16     value = float("inf")
17     for move in [1, 2]:
18         if move <= stones:
19             value = min(value, alphabeta(
20                 stones - move, alpha, beta, True))
21             beta = min(beta, value)
22             if beta <= alpha:
23                 break
24     return value
```

Enter number of stones: 7

Stones left: 7
Computer takes: 1
Your move (1 or 2): 2

Stones left: 4
Computer takes: 1
Your move (1 or 2): 2

Stones left: 1
Computer takes: 1
Computer wins!

PS C:\Users\soum\Downloads\New folder (2)>

Result

The game was successfully implemented using Alpha-Beta pruning.

Conclusion

Alpha-Beta pruning reduces unnecessary searches in the Minimax algorithm and helps the computer choose the best move efficiently.